



COMPUTER PROGRAMMING MID-II QUESTION BANK

SHORT ANSWER TYPE QUESTIONS:

1. What is a set? Write one example.

Ans: A Set is an unordered (un-indexed) collection of immutable objects that does not allow duplicates.

Example:

```
basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}  
print(basket)
```

Output:

```
{'orange', 'apple', 'pear', 'banana'}
```

2. Write the difference between `remove()` and `discard()` methods in set.

Ans: The **`remove()`** method removes a specified element from a set. If the element is not found, it raises a **`KeyError`**.

Example:

Example1:	Example2:
<pre>s = {1, 2, 3} s.remove(2) print(s)</pre>	<pre>s = {1, 2, 3} s.remove(4) print(s)</pre>
Output: <pre>{1, 3}</pre>	Output: <code>KeyError: 4</code>

The **`discard()`** method also removes a specified element but does nothing if the element is not found. No error is raised.

Example:

```
s = {1, 2, 3}  
s.discard(2)  
print(s)  
s.discard(4)  
print(s)
```

Output:



{1, 3}

{1, 3}

3. What is a dictionary in Python? Give one simple example.

Ans: A dictionary contains a collection of indices, which are called keys, and a collection of values. Each key is associated with a single value. The association of a key and a value is called a key-value pair or sometimes an item.

Example:

```
my_dict = {'apple': 3, 'banana': 5, 'cherry': 5}
print(my_dict)
```

Output: {'apple': 3, 'banana': 5, 'cherry': 5}

4. Define a class in Python.

Ans: A class is a collection of objects. Classes are blueprints for creating objects. A class defines a set of attributes and methods that the created objects (instances) can have.

5. What is the use of constructors in OOP?

Ans: In Object-Oriented Programming (OOP), a **constructor** is a special method used to initialize objects when they are created.

6. What is inheritance?

Ans: Inheritance is used to inherit the properties and behaviours of one class to another. The class that inherits another class is called a **child class** and the class that gets inherited is called a **base class or parent class**.

7. Define abstraction.

Ans: Abstraction is a process of handling complexity by hiding unnecessary details from the user and providing only necessary features or functionality.

8. What is encapsulation?

Ans: Encapsulation in Python means binding variables and methods together within a class to restrict direct access to data. Python encapsulation helps make programs more secure, modular, and easier to manage.



Need of Encapsulation:

- Encapsulation in Python keeps internal data safe from accidental changes.
- It allows controlled access through getter and setter methods.
- It improves code readability and maintainability.
- It helps us create reusable and flexible class structures.

9. What is polymorphism?

Ans: The word "polymorphism" means "many forms", and in programming it refers to methods/functions/operators with the same name that can be executed on many objects or classes.

10. What is a file path?

Ans: A **file path** is the location of a file or folder in a computer's storage system. It tells the operating system where a file is stored so that Python program can find it.

11. What is append mode in file handling?

Ans: File is in write mode only; new data will be added to the end of existing data i.e. no overwriting. If file does not exist it is created.

12. What is exception handling?

Ans: Exception Handling is a mechanism in programming that allows a program to detect and handle errors during runtime without crashing

13. What is the purpose of `with open()`?

Ans: The purpose of using **with open()** in Python file handling is to automatically manage resources especially opening and closing files safely and cleanly.

Example:

Reading a file:

```
with open("data.txt", "r") as file:  
    content = file.read()  
print(content)
```



14. What is an exception in Python?

Ans: An exception is an error that occurs while a program is running, interrupting the normal flow of execution.

15. What is a module?

Ans: ~~A Module allows you to logically organize your Python code. When code gets to be large enough, the tendency is to break it up into organized pieces that can still interact with one another at a functioning level.~~

- These pieces generally have attributes that have some relation to one another.
- These pieces should be shared, so Python allows a module the ability to “bring in” and use attributes from other modules to take advantage of work that has been done, maximizing **code reusability**.

PART-2: LONG ANSWER TYPE QUESTIONS:

1) Explain set operations with examples.

Ans:

The Set container also supports set like operations such as union (|), intersection (&), difference (-) and symmetric difference (^). These are based on simple Set theory.

➤ UNION (|):

The Union of two sets represents the combination of all the values in the two sets.

Example:

```
s1 = {'apple', 'orange', 'banana'}  
s2 = {'grapefruit', 'lime', 'banana'}  
print('Union:', s1 | s2)
```

Output:

```
Union: {'orange', 'apple', 'lime', 'banana', 'grapefruit'}
```

➤ INTERSECTION (&):

The intersection of two sets represents the common values between two sets.

Example:

```
s1 = {'apple', 'orange', 'banana'}  
s2 = {'grapefruit', 'lime', 'banana'}
```



```
print('Intersection:', s1 & s2)
```

Output:

Intersection: {'banana'}

➤ **DIFFERENCE (-):**

The difference between two sets is the set of values in the first set that are not in the second set.

Example:

```
s1 = {'apple', 'orange', 'banana'}  
s2 = {'grapefruit', 'lime', 'banana'}  
print('Difference:', s1 - s2)
```

Output:

Difference: {'orange', 'apple'}

➤ **SYMMETRIC DIFFERENCE (^):**

The symmetric difference represents all the unique values in the two sets (that is it is the inverse of the intersection).

Example:

```
s1 = {'apple', 'orange', 'banana'}  
s2 = {'grapefruit', 'lime', 'banana'}  
print('Symmetric Difference:', s1 ^ s2)
```

Output:

Symmetric Difference: {'lime', 'grapefruit', 'orange', 'apple'}

2) Explain dictionary operations with examples.

Ans:

A dictionary contains a collection of indices, which are called **keys**, and a collection of values. Each key is associated with a single value. The association of a key and a value is called a **key-value pair** or sometimes an **item**.

Accessing Items via Keys:

- You can access the values held in a Dictionary using their associated key. This is specified using either the square bracket ([]) notation (where the key is within the brackets) or the get() method:

```
my_dict = {'apple': 3, 'banana': 5, 'cherry': 5}
```



```
print(my_dict['apple'])
```

Output: 3

Using get():

```
my_dict.get('banana')
```

Output:5

Adding a New Entry:

- A new entry can be added to a dictionary by providing the key in square brackets and the new value to be assigned to that key:

```
my_dict = {'apple': 3, 'banana': 5, 'cherry': 5}
my_dict ['Guava']=15
print(my_dict)
```

Output: {'apple': 10, 'banana': 5, 'cherry': 5, 'Guava': 15}

Changing a Keys Value:

- The value associated with a key can be changed by reassigning a new value using the square bracket notation.

```
my_dict = {'apple': 3, 'banana': 5, 'cherry': 5}
my_dict['apple']=100
print(my_dict)
```

Output: {'apple': 100, 'banana': 5, 'cherry': 5}

Removing an Entry:

- An entry into the dictionary can be removed using one of the methods **pop()** or **popitem()** method or the **del** keyword.

Using pop() method:

- The pop() method removes the entry with the specified key. **This method returns the value of the key being deleted.**
- If the key is not present error will be generated.

```
my_dict = {'apple': 3, 'banana': 5, 'cherry': 5}
my_dict.pop('apple')
```

Output: 3

```
print(my_dict)
```



Output: {'banana': 5, 'cherry': 5}

Using popitem() method:

- The popitem() method removes the last inserted item in the dictionary.
- **The key:value pair being deleted is returned from the method.**

```
my_dict = {'apple': 3, 'banana': 5, 'cherry': 5}
my_dict.popitem()
```

Output: ('cherry', 5)

Using del keyword:

- del keyword removes the entry with the specified key from the dictionary.
- This keyword just deletes the item; **it does not return the associated value.**
- It is potentially more efficient than pop()

```
my_dict = {'apple': 3, 'banana': 5, 'cherry': 5}
del my_dict['apple'] ## (it does not return the value)
print(my_dict)
```

Output: {'banana': 5, 'cherry': 5}

3) Explain in detail about List comprehensions in python.

Ans:

List Comprehensions:

- It is a very powerful mechanism that can be used to generate new lists.

Syntax:

- Syntax for List Comprehension is
[<expression> **for** item **in** iterable <**if** optional_condition>]

Note: Whole **for** statement and expression is surrounded by the square brackets normally used to create a **List**.

- When a List Comprehension is executed, it generates a new list by applying the expression to the items in another collection.
- For example, given one list of integers(list1), another list(list2) can be created using the **List Comprehension format**:

```
list1 = [1, 2, 3, 4, 5, 6]
print('list1:', list1)
list2 = [item for item in list1]
```



```
print('list2:', list2)
```

Output: list1: [1, 2, 3, 4, 5, 6]

list2: [1, 2, 3, 4, 5, 6]

- This feature is not limited to processing values in a list; any iterable collection can be used such as **Tuples** or **Sets** as the source of the values to process.
- Another feature of the List Comprehension is the ability to filter the values passed to the expression using the **optional if condition**.
- For example, if we wish to only consider even numbers for the expression then we can use the optional if statement to filter out all odd numbers:

```
list3 = [item + 1 for item in list1 if item % 2 == 0]  
print('list3:', list3)
```

Output: list3: [3, 5, 7]

- Thus only the even numbers were passed to the expression `item + 1` resulting in only the values 3, 5 and 7 being generated for the new list.

Applications of List Comprehension:

- List comprehension is a programming construct used to efficiently create new lists by transforming, filtering, or combining items from existing iterables in a concise, readable way.

1. **Filtering Elements:** Extract items from a list that meet certain criteria. For example, creating a list of fruits containing the letter "a".

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]  
newlist = [x for x in fruits if "a" in x]  
print(newlist)
```

Output: ['apple', 'banana', 'mango']

2. **Transforming Data:** Apply an operation or function to each element. For instance, squaring numbers.

```
numbers = [2, 3, 4, 5]  
squares = [n ** 2 for n in numbers]  
print(squares)
```

Output: [4, 9, 16, 25]



3. **Nested Comprehensions:** Build complex structures like matrices.. For example, a multiplication table:

```
multiplication = [[i * j for j in range(1, 6)] for i in range(2, 5)]  
print(multiplication)
```

Output: `[[2, 4, 6, 8, 10], [3, 6, 9, 12, 15], [4, 8, 12, 16, 20]]`

4. **Combining Iterables:** Generate coordinate pairs or combinations, such as
- ```
coordinates = [(x, y) for x in range(3) for y in range(3)]
print(coordinates)
```

**Output:** `[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]`

#### 4) Explain any two OOP concepts with examples.

**Ans:**

##### **ENCAPSULATION:**

Encapsulation in Python means binding variables and methods together within a class to restrict direct access to data. Python encapsulation helps make programs more secure, modular, and easier to manage.

##### **Need of Encapsulation:**

- Encapsulation in Python keeps internal data safe from accidental changes.
- It allows controlled access through getter and setter methods.
- It improves code readability and maintainability.
- It helps us create reusable and flexible class structures.

##### **Access Modifiers:**

In Python, encapsulation is achieved through the use of access modifiers. These access modifiers control the visibility of class attributes and methods from outside the class. The three common access modifiers are:

- **Public:** Members are accessible from outside the class.
- **Protected:** Members are accessible within the class and its subclasses.
- **Private:** Members are only accessible within the class.

##### **1. Public Members in Class**

- Public members in a class are attributes or methods that can be accessed from anywhere outside the class.



- In the following program, we have a Student class with two attributes: name and age. These attributes can be accessed from outside the class. Therefore name and age attributes in this example are public members.

**Program:**

```
class Student:
 def __init__(self, name, age):
 self.name = name
 self.age = age

student = Student("Green", 20)
print(student.name) # Accessing public attribute
print(student.age)
```

**Output:**

```
Green
20
```

**2. Protected Members in Class**

- Protected members in a class are attributes or methods that can be only be accessed from inside the same class, and from inside sub-classes.
- In the following program, we have two classes: Animal and Dog. Dog class is a subclass of Animal. Inside the Animal class, we have an attribute `_type`, which is protected.

**Example:**



```
class Animal:
 def __init__(self):
 self._type = "Unknown"

class Dog(Animal):
 def __init__(self):
 super().__init__()
 self._type = "Dog"

dog = Dog()
print(dog._type)
```

Dog

### 3. Private Member in Class

- Private members in a class are attributes or methods that can be only be accessed from inside the same class.
- In the following program, we have a class: BankAccount. BankAccount class has an attribute `__balance`, which is a private member.

#### Example:

```
class BankAccount:
 def __init__(self):
 self.__balance = 0

 def deposit(self, amount):
 self.__balance += amount

 def get_balance(self):
 return self.__balance

account = BankAccount()
account.deposit(50000)
print("balance:", account.get_balance()) # Accessing private method

balance: 50000
```



## **INHERITANCE:**

Inheritance allows us to define a class that inherits all the methods and properties from another class.

Parent class is the class being inherited from, also called base class.

Child class is the class that inherits from another class, also called derived class.

### **Creating a Parent Class**

The class whose attributes and methods are inherited is called as parent class. It is defined just like other classes i.e. using the class keyword.

#### **Syntax:**

```
class ParentClassName:
 {class body}
```

### **Creating a Child Class**

Classes that inherit from base classes are declared similarly to their parent class, however, we need to provide the name of parent classes within the parentheses.

#### **Syntax:**

```
class SubClassName (ParentClass1[, ParentClass2, ...]):
 {sub class body}
```

### **Example**

Create a class named Dog, which will inherit the properties and methods from the Animal class:



```
class Animal:
 name = ""
 def eat(self):
 print("I can eat")

inherit from Animal
class Dog(Animal):
 def display(self):
 print("My name is ", self.name)

labrador = Dog()
labrador.name = "Jimmy"
labrador.eat()

call subclass method
labrador.display()
```

**Output:**

```
I can eat
My name is Jimmy
```

**5) Write a program to define a class Student and display details using a method.**

**Ans:**

**Program:**



```
class Student:

 def __init__(self, name, roll_number, dept):

 self.name = name # Instance attribute 1
 self.roll_number = roll_number # Instance attribute 2
 self.dept = dept # Instance attribute 3

 def display_details(self):

 print("Student Details:")
 print(f"Name: {self.name}")
 print(f"Roll Number: {self.roll_number}")
 print(f"Dept: {self.dept}")
 print("-" * 30)

student1 = Student(name="John", roll_number="A101", dept="AIML")
print("Displaying Details for Student 1")
student1.display_details()

student2 = Student(name="Bob", roll_number="C205", dept="CSE")
print("\nDisplaying Details for Student 2")
student2.display_details()
```

### Output:

```
Displaying Details for Student 1
Student Details:
Name: John
Roll Number: A101
Dept: AIML

Displaying Details for Student 2
Student Details:
Name: Bob
Roll Number: C205
Dept: CSE

```



## 6) Write a Python program to demonstrate constructor usage.

**Ans:**

To understand the meaning of classes we have to understand the built-in `__init__()` method.

All classes have a method called `__init__()`, which is always executed when the class is being initiated.

Use the `__init__()` method to assign values to object properties, or other operations that are necessary to do when the object is being created:

```
class Book:

 def __init__(self, title, author):

 self.title = title
 self.author = author
 def display_info(self):

 print("-----")
 print(f"Title: {self.title}")
 print(f"Author: {self.author}")

book1 = Book("Think Python", "Allen Downey")

book2 = Book("Learning Python ", "Mark Lutz")

print("\nDisplaying Book Information")
book1.display_info()
book2.display_info()
```

**Output:**

```
--- Displaying Book Information ---

Title: Think Python
Author: Allen Downey

Title: Learning Python
Author: Mark Lutz
```



**7) Write a Python program using inheritance (Parent → Child).**

**Ans:**

Inheritance allows us to define a class that inherits all the methods and properties from another class.

**Parent class** is the class being inherited from, also called base class.

**Syntax:**

```
class ParentClassName:
 {class body}
```

**Child class** is the class that inherits from another class, also called derived class.

**Syntax:**

```
class SubClassName (ParentClass1[, ParentClass2, ...]):
 {sub class body}
```

**Program:**

Create a class named Dog, which will inherit the properties and methods from the Animal class:

```
class Animal:
 def __init__(self, name):
 self.name = name

 def info(self):
 print("Animal name:", self.name)

class Dog(Animal):
 def sound(self):
 print(self.name, "always barks")

d = Dog("Alex")
d.info() # Inherited method
d.sound()
```



**Output:**

Animal name: Alex  
Alex always barks

**8) Explain method overriding with a suitable example.**

**Ans:**

Method overriding is a feature of OOP where a child class provides its own version of a method that is already defined in the parent class. The method name and parameters remain the same, but the behavior changes. It is used when a subclass wants to modify or extend the parent class method.

```
class Animal:
 def sound(self):
 print("Animals make sounds")
class Dog(Animal):
 def sound(self): # overriding the parent's method
 print("Dog barks")
Creating objects
a = Animal()
d = Dog()
a.sound() # Output: Animals make sounds
d.sound() # Output: Dog barks
```

```
Creating objects
s1 = Shape()
s2 = Square()

Calling methods
s1.sides() # (Parent class method)
s2.sides() # (Child class overrides the method)
```

**Output:**

This shape has many sides  
A square has 4 sides



**9) Explain the difference between method overloading and overriding.**

**Ans:**

| <b>Method Overloading</b>                                                                                                             | <b>Method Overriding</b>                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>Occurs within a single class</li></ul>                                                          | <ul style="list-style-type: none"><li>Occurs between a parent and child class</li></ul>                                                                |
| <ul style="list-style-type: none"><li>Involves multiple methods with the same name but different parameters</li></ul>                 | <ul style="list-style-type: none"><li>Involves a method in the child class with the same name and parameters as a method in the parent class</li></ul> |
| <ul style="list-style-type: none"><li>Decision is made at compile-time (static binding)</li></ul>                                     | <ul style="list-style-type: none"><li>Decision is made at runtime (dynamic binding)</li></ul>                                                          |
| <ul style="list-style-type: none"><li>The return type can be the same or different</li></ul>                                          | <ul style="list-style-type: none"><li>The return type must be the same or a subtype in the child class</li></ul>                                       |
| <ul style="list-style-type: none"><li>No relationship required between the overloaded methods</li></ul>                               | <ul style="list-style-type: none"><li>Requires an inheritance relationship between the parent and child classes</li></ul>                              |
| <ul style="list-style-type: none"><li>Overloaded methods can have different access modifiers</li></ul>                                | <ul style="list-style-type: none"><li>Overriding methods must have the same or higher access modifier in the child class</li></ul>                     |
| <ul style="list-style-type: none"><li>Overloading allows methods with different functionality to have the same name</li></ul>         | <ul style="list-style-type: none"><li>Overriding allows child classes to provide specific implementations for inherited methods</li></ul>              |
| <ul style="list-style-type: none"><li>Method parameters can vary in type, order, or number</li></ul>                                  | <ul style="list-style-type: none"><li>Method parameters must have the same type, order, and number in both parent and child classes</li></ul>          |
| <ul style="list-style-type: none"><li>Method overloading is achieved within a single class by changing the method signature</li></ul> | <ul style="list-style-type: none"><li>Method overriding is achieved by using the <code>@Override</code> annotation in the child class</li></ul>        |
| <ul style="list-style-type: none"><li>Overloading is a form of compile-time polymorphism</li></ul>                                    | <ul style="list-style-type: none"><li>Overriding is a form of runtime polymorphism</li></ul>                                                           |



**10) Explain file reading techniques: read(), readline(), readlines().**

**Ans:**

**READING FROM A FILE:**

Read from file python provide many functions like:

▪ **Reading an Entire file (Read()):**

We can read an entire file using the keyword read().

**Syntax:** Filehandle.read()

**Example:**

**sample.txt:**

Python is an interpreted Language.

Python is a high-level, general-purpose programming language known for its clear, readable syntax.

It was created by Guido van Rossum and first released in 1991.

**Program:**

```
fp = open("C:/Users/User/Documents/sample.txt", 'r')
content = fp.read()
print (content)
fp.close()
```

Python is an interpreted Language.

Python is a high-level, general-purpose programming language known for its clear, readable syntax.

It was created by Guido van Rossum and first released in 1991.

**Readline():**

We can read a line of input using keyword readline().

**Syntax:** Filehandle.readline()

**Example:**

**Program:**



```
fp = open("C:/Users/User/Documents/sample.txt", 'r')
content = fp.readline()
k= fp.readline()
print (content)
print(k)
fp.close()
```



Python is an interpreted Language.

Python is a high-level, general-purpose programming language known for its clear, readable syntax.

### **Readlines():**

Readlines() Reads all lines and returns them in a list.

**Syntax:** Filehandle.readlines()

```
fp=open("C:/Users/admin/OneDrive/sample.txt", "r")
content=fp.readlines()
print(content)
```

```
['Python is an interpreted Language.\n', 'Python is a high-level, general-purpose\nprogramming language known for its clear, readable syntax.\n', 'It was created by\nGuido van Rossum and first released in 1991.']
```

### **11) Explain the different types of errors in python.**

**Ans:**

#### **TYPES OF ERRORS:**

##### **1. Syntax Errors**

These occur when the code violates the grammatical rules of the Python language. The Python interpreter detects these during the parsing phase, and the program will not run.

Description: Analogous to grammatical mistakes in human language.

#### **Examples:**

- Forgetting a colon (:) at the end of a def, if, for, or while statement.



```
if x == 10
print("x is 10")
```

```
Cell In[7], line 1
 if x == 10
 ^
SyntaxError: expected ':'
```

- Mismatched parentheses or quotes.

**IndentationError:** Incorrect indentation, which is significant in Python (can lead to an IndentationError or TabError).

**Example:**

```
if x == 10:
print("x is 10")
```

**Output:**

```
print("x is 10")
^
```

**IndentationError:** expected an indented block after 'if' statement on line 1

## 2. Runtime Errors (Exceptions)

These errors occur while the program is executing, even if the syntax is correct. They halt the program's execution if not handled properly. These errors cannot be detected until the code is actually run.

**Examples:**

**ZeroDivisionError:** This exception is thrown whenever a division by zero is attempted.

**Example:**

```
a=5
b=0
print(a/b)
```

```

ZeroDivisionError
```



**NameError:** When a variable (or) function name cannot be found in the current scope, then exception NameError is thrown.

**Example:**

```
a=5
b=6
print(a/B)
```

-----  
**NameError**

**TypeError:** When an operation (or) function is done to an object of the incorrect type, such as by adding a string to an Integer, an exception is thrown.

**Example:**

```
a=5
b="abc"
print(a/b)
```

-----  
**TypeError**

**IndexError:** This exception is thrown when a list, tuple (or) other sequence type's index is outside of borders.

**Example:**

```
a=[1,3,5.7]
print(a[4])
```

-----  
**IndexError**

**KeyError:** When a key cannot be found in a dictionary, this exception is thrown.

**Example:**

```
a={1:"a",2:"b",3:"c"}
print(a[4])
```

-----  
**KeyError**



**Value error:** This exception is thrown when an invalid argument or input is passed to a function. An example would be trying to convert a string to an integer when the string does not represent a valid integer.

**Example:**

```
a="abcde"
print(float(a))
```

-----  
**ValueError**

**FileNotFoundError (IOError):** Trying to open a file that doesn't exist.

Example:

```
fp = open("C:/Users/User/Documents/kample.txt",'r')
content = fp.readlines()
for line in content:
 print(line)
```

-----  
**FileNotFoundError** Traceback (most recent call last):

### 3. Logical Errors (Bugs)

These are the most challenging to find because the code runs without crashing and without raising an error message. However, the program produces an incorrect or unexpected result.

Description: The code's logic is flawed. The program does what you told it to do, not what you intended it to do.

**Examples:**

- Using the wrong mathematical formula or algorithm.
- Incorrect logic in an if statement that leads to the wrong path of execution.
- Making a mistake in a mathematical calculation, such as using subtraction instead of addition.



**12) Explain briefly about try-except statement with an example.**

**Ans:**

Try -Except Statements: The try and except statements are used to handle runtime errors.

**Syntax:**

```
try :
 statements
except :
 statements
```

The try statement works as follows:-

- First, the try clause (the statement(s) between the try and except keywords) is executed.
- If no exception occurs, the except clause is skipped and execution of the try statement is finished.
- If an exception occurs during execution of the try clause, the rest of the clause is skipped. Then if its type matches the exception named after the except keyword, the except clause is executed, and then execution continues after the try statement.

**Example:**

```
X=int(input("Enter the value of X:"))
Y=int(input("Enter the value of Y:"))
try:
 result = X / (X-Y)
 print("result=".result)
except ZeroDivisionError:
 print("Division by Zero")
```



**Output:**

|                         |                         |
|-------------------------|-------------------------|
| Enter the value of X:10 | Enter the value of X:10 |
| Enter the value of Y:10 | Enter the value of Y:5  |
| Division by Zero        | result= 2.0             |

**13) Explain briefly about try-except-finally statement with an example.**

**Ans:**

**Try –Except-Finally:**

- A finally clause is always executed before leaving the try statement, whether an exception has occurred or not.
- The finally clause is also executed “on the way out” when any other clause of the try statement is left via a break, continue or return statement.

**Syntax:**

```
try:
 Statements
except:
 Statements
finally:
 Statements
```

**Example:**

```
X=int(input("Enter the value of X:"))
Y=int(input("Enter the value of Y:"))
try:
 result = X / (X-Y)
except ZeroDivisionError:
 print("Division by Zero")
else:
 print("result=",result)
finally:
 print ("executing finally clause")
```



**Output:**

|                          |                          |
|--------------------------|--------------------------|
| Enter the value of X:10  | Enter the value of X:10  |
| Enter the value of Y:5   | Enter the value of Y:10  |
| result= 2.0              | Division by Zero         |
| executing finally clause | executing finally clause |

**14) Write a program to read a text file and count words and lines.**

**Ans:**

**Text file:**

```
Python is an interpreted language.
Python is a high-level programming language.
This is an example to create a file in python.
```

**Program:**

```
fp = open("C:/Users/User/Documents/sample.txt",'r')
lines = 0
word = 0
for line in fp:
 words = line.split ()
 lines= lines+1
 word = word + len(words)
print("Number of line", lines)
print("Number of words", word)
```

```
Number of line 3
Number of words 21
```

**15) Explain how to create and import modules in Python with examples.**

**Ans:**

**Creating and Managing Modules:**

**To create a module:**

1. Write your functions, variables, or classes in a .py file. For example, create a file called module1.py:



---

**Example:**

```
def sample(name):
 print("hello", name)
```

**Importing and Using a Module**

To use your module in another script, simply import it.

**Example:**

```
import module1
module1.sample("John")
```

**Output:**

Hello John

**You can also import specific items:**

- It allows us to import specific attributes **from** a module into the current namespace.

➤ **Example:**

```
From module1 import sample
Sample("John")
```

**Or use an alias:**

```
import module1 as jv
jv.sample("John")
```